

Computing with Finite Observations of Non-Constructive Objects

A Verified Ultrafilter Interface

Krzysztof Woś

The University of Tokyo
Graduate School of Information Science and Technology
Department of Creative Informatics

Abstract

Can a finite program compute with infinitesimals while leaving their non-constructive mathematical interpretation unspecified? We study an interface that records only observations made during execution. Each accepted observation preserves the existence of a free-ultrafilter completion. Lean 4 proofs connect an executable periodic Laurent language to these completions and interpret each accepted finite adaptive run using one fixed completion. Standard-part extraction is sound and complete for a value shared by all remaining completions. Every such finite real value is rational and is found by the extractor. A generic compiler builds literal difference quotients for arbitrary rational polynomials at rational points with steps c/n^k , where $c \neq 0$ and $k \geq 1$. The extractor returns the ordinary derivative. A second, symbolic compiler handles finite vectors, general division, and elementary functions on explicit smooth domains. Lean proves that its Jacobian-vector products are standard parts of literal quotients for every compatible completion with a nonzero infinitesimal step. An adaptive example makes an observation discharge a necessary division condition: an infinitesimal step may be zero in some completions, so the program either establishes its nonzeroness or selects a fallback. Both accepted branches extract 12 for the cubic at 2. The implementation retains exact finite syntax without constructing an ultrafilter or truncating a series. Kernel replay checks exported instances. The result is an operational bridge for a restricted language. General sequence predicates and verified capture of arbitrary Python computations remain outside it.

Keywords Non-constructive mathematics, finite observation, infinitesimals, ultrafilters, operational semantics, Lean.

1 Introduction

A classical existence theorem can justify a mathematical object without supplying executable data for it. This raises an operational question: what may a finite program do with such an object? Hyperreal arithmetic provides a concrete setting. Its ultrapower interpretation uses a free ultrafilter, while a particular calculation may inspect only a few comparisons and return an ordinary real number. Computing those observations need not determine the entire ultrafilter.

We investigate this separation through an executable interface. The program builds exact expressions for real sequences and records selected comparison sets. The runtime accepts a selection only after checking that the retained recurring indices remain infinite. A classical extension theorem then supplies a free ultrafilter containing the observations. This existence proof justifies the execution without becoming an algorithm that constructs its witness.

There are two distinct guarantees. First, an accepted execution has a compatible completion, so its choices can be read as observations of one mathematical object. Second, standard-part extraction returns exactly the finite values shared by every completion compatible with that execution. The first permits underdetermined choices. The second permits an ordinary result

before all choices have been made. For example, $(-1)^n$ has completion-dependent sign, but $1 + (-1)^n/n$ has standard part 1 in every free completion.

The fact that a program makes finitely many observations is not sufficient for an algorithm. Those observations concern actual subsets of $\mathbb{N}$, and their consistency can itself require mathematical reasoning. Propositional satisfiability of opaque names does not establish it. We therefore distinguish the general semantic requirement from an executable instance: finite periodic rational coefficients, n , $1/n$, arithmetic, and division by nonzero rational monomials. On this language, exact normalization and comparison masks discharge the required obligations. Unsupported sequence predicates are outside this implementation.

The paper develops three connected results:

1. A specification of finite observations and classical completion, with an explicit separation between existential consistency and executable decisions.
2. Verified normalization, comparison, and state refinement for a periodic Laurent language, with sound and complete extraction of completion-invariant standard parts and fixed-completion semantics for accepted adaptive programs.
3. Generic executable infinitesimal differentiation for rational polynomials, a symbolic derivative compiler for elementary vector functions, and an adaptive observation that establishes a necessary denominator condition. The quotient semantics applies to either accepted step choice.

The extension theorem is classical supporting mathematics. The calculus identities are elementary. Their role is to make the connection between finite execution and a non-constructive interpretation precise and inspectable. We do not claim a new construction of the hyperreals, a general algorithm for ultrafilter membership, or a performance advantage over symbolic algebra.

2 An Adaptive Infinitesimal Calculation

Let $\varepsilon(n) = 1/n$, with any convention at $n = 0$. Changing the convention at this single index does not affect any free-ultrafilter interpretation. For every positive real a , the inequality $0 < \varepsilon(n) < a$ holds eventually. Consequently, in every free completion the represented quantity is positive, nonzero, and smaller than every positive real constant. Its standard part is zero. Standard part zero does not make the quantity itself zero.

Take $f(x) = x^3$, $x = 2$, and define actual increments

$$\Delta x = \varepsilon, \quad \Delta y = f(2 + \varepsilon) - f(2), \quad Q = \frac{\Delta y}{\Delta x}.$$

The denominator is nonzero in each completion, so the quotient is meaningful. Exact algebra at every positive index gives

$$Q(n) = \frac{(2 + 1/n)^3 - 8}{1/n} = 12 + \frac{6}{n} + \frac{1}{n^2}. \quad (1)$$

Thus $Q(n) > 12$ at every positive index, while $Q(n) - 12 \rightarrow 0$. In every free completion,

$$Q > 12, \quad \text{st}(Q) = 12 = f'(2).$$

Here $Q > 12$ means that the set of indices satisfying the inequality belongs to the completion. The limit statement uses the filter definition of standard part in Section 3.

This example makes the fraction interpretation precise. The finite increments Δy and Δx have a literal quotient. Its standard part is the derivative. The quotient is not equal to the derivative before taking standard part. No rule allowing arbitrary manipulation of differential notation is assumed.

The public interface expresses the original quotient directly:

```
from hyperreals import LeanResidueSystem

system = LeanResidueSystem()
epsilon = system.infinitesimal()
x = system.constant(2)
quotient = ((x + epsilon)**3 - x**3) / epsilon
assert quotient.standard_part() == 12
```

The returned value is an exact rational. The expression is sent to the verified normalizer without a precomputed derivative or limit certificate. Lean proves the sequence identity, positivity and infinitesimal properties, the completion-relative limit, and the ordinary derivative in `InfinitesimalCase`. The theorem `quotientExpr_denote` connects the actual expression tree to the stated quotient, and `quotient_extracts` checks the executable result by kernel reduction. For a literal field interpretation, `hyperreal_quotient_standardPart` proves the same standard-part equality in Mathlib’s existing hyperreal field, using its distinguished positive infinitesimal. The companion strict-inequality theorem proves that this field element is greater than 12. The script `scripts/infinitesimal_case.py` runs the public API and exports replayable instances.

2.1 An Observation Establishes the Domain

The forward calculation alone requires no discretionary observation. To make the observation mathematically necessary, instead consider

$$b(n) = \begin{cases} 0 & n \text{ even,} \\ 1 & n \text{ odd,} \end{cases} \quad h = b\varepsilon, \quad g = bn.$$

The step h is infinitesimal in every free completion. It is exactly zero in completions selecting even indices and nonzero in those selecting odd indices. The represented reciprocal g satisfies $hg = 1$ only in the latter. The runtime can represent g using existing multiplication and a periodic table, but this fact does not add unrestricted division to the language.

The program queries $h = 0$. A true answer retains even indices and replaces h by the fallback $\delta = \varepsilon^2$. A false answer retains odd indices, keeps $\delta = h$, and licenses multiplication by g as division by h . Figure 1 shows the two branches. The observation is a caller-proposed polarity checked against the remaining support, not evaluation at a sampled index or membership testing in a preselected ultrafilter.

Corollary 1 (Observation-dependent derivative extraction). *Every accepted execution has one fixed compatible free completion. In every completion of its recorded trace, the selected step δ is nonzero and infinitesimal, its returned expression denotes the literal quotient $((2 + \delta)^3 - 8)/\delta$, and that quotient has standard part 12. The executable extractor returns 12 on both branches. On the positive branch, the original h has no multiplicative inverse.*

Proof. The accepted observation gives eventual equality $h = 0$ or eventual inequality $h \neq 0$ in the same completion. On the negative branch, the periodic indicator takes value 1, so $hg = 1$. On the positive branch, replacing h by ε^2 establishes a nonzero infinitesimal denominator. The two resulting expression trees extract 12 by kernel reduction. Fixed-completion realization then applies to the entire selected run. The declarations `selected_step_nonzero`, `selected_step_infinitesimal`, `accepted_quotient_correspondence`, `accepted_result`, and `zero_branch_has_no_inverse` in `DomainInfinitesimal` prove these obligations. $\square$

Before the observation, the multiplication expression $((2 + h)^3 - 8)g$ has standard part 0 in even completions and 12 in odd completions. It therefore has no shared finite standard

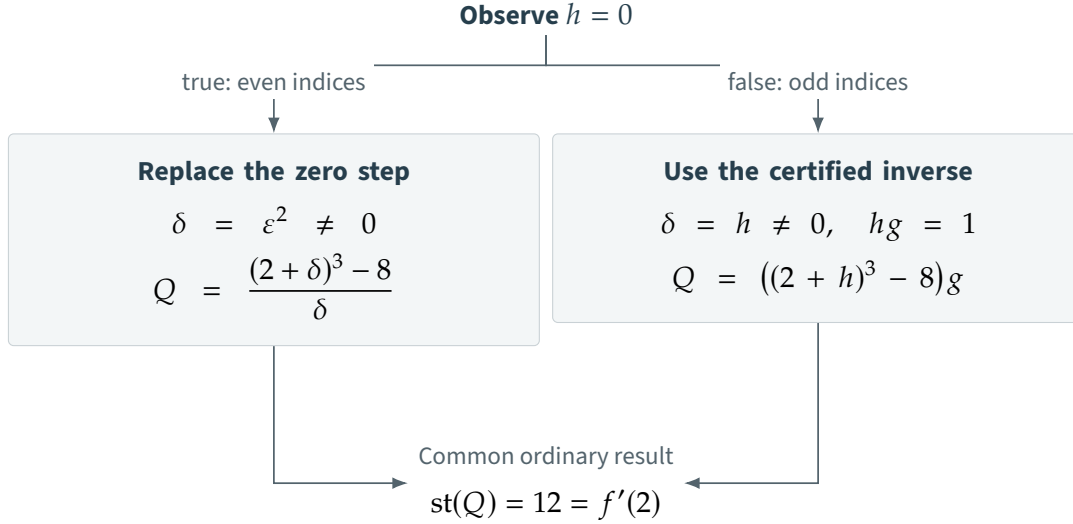


Figure 1. The observation supplies a division obligation. Each accepted branch uses a nonzero infinitesimal step and has its own compatible free completions. The inverse identity $hg = 1$ is valid in every completion of the negative branch. It is false in every completion of the positive branch, where the step is replaced.

part, as proved by `guarded_no_common_standardPart`. The zero branch cannot be repaired by cancelling h . Its fallback changes the step. This is the distinction between an infinitesimal quantity and an invertible infinitesimal quantity that the observation enforces.

The script `scripts/domain_infinitesimal_case.py` captures the two branches in fresh systems and exports their traces and expressions for kernel replay. Tests separately evaluate the formulas with rational arithmetic and check that the opposite observation is rejected after a branch has been selected. These concrete checks do not establish a compiler-correctness theorem for Python. The corollary quantifies over all accepted answer streams for the explicit Lean program.

2.2 Generic Polynomial Differentiation

The cubic is not the limit of the arithmetic result. A second compiler accepts an arbitrary finite list of rational coefficients and constructs the Horner expression for its polynomial. Given $P \in \mathbb{Q}[X]$, $a, c \in \mathbb{Q}$, $c \neq 0$, and $k \in \mathbb{N}$ with $k \geq 1$, it forms the original syntax

$$h(n) = \frac{c}{n^k}, \quad Q_{P,a,c,k} = \frac{P(a+h) - P(a)}{h}.$$

No derivative value, expanded numerator, or convergence certificate is supplied to this compiler.

Theorem 1 (Executable polynomial differentiation). *For every such polynomial, point, scale, and exponent, and every nonempty residue support S , the generated expression denotes the literal quotient at positive indices and*

$$\widehat{\text{st}}_S(Q_{P,a,c,k}) = P'(a).$$

The returned rational agrees with the usual real derivative. Its standard part is $P'(a)$ in every compatible free completion.

Proof. A recursive polynomial divided difference $D_{P,a}(h)$ satisfies the exact identity

$$hD_{P,a}(h) = P(a+h) - P(a)$$

even when $h = 0$. Writing $P(X) = p_0 + XR(X)$ gives the executable recursion

$$D_{P,a}(h) = R(a + h) + aD_{R,a}(h).$$

The same recursion proves convergence to $P'(a)$ whenever $h \rightarrow 0$, and an independent differentiation proof identifies that value with the ordinary derivative. The specified c/n^k is nonzero at positive indices, so cancellation identifies the divided difference with the literal quotient. Completeness of extraction, proved in Section 4, converts this convergence proof into the claimed executable result on every nonempty support. $\square$

The proofs are in `PolynomialDifferentiation`. The declaration `rational_polynomial_representable` covers every rational polynomial with a finite coefficient list. The declarations `quotientExpr_denote` and `quotientExpr_computes_derivative` connect the generated expression, its executable extraction, and the usual derivative. The more general `dividedDifference_standardPart_of_standardPart` requires only that the executable extractor return zero for the increment on the current support. It therefore handles represented increments whose infinitesimality becomes established after an observation, including increments that are zero on some active residues. That polynomial expression remains meaningful at zero. Calling it a literal quotient still requires a separate nonzero condition, exactly the obligation isolated by the adaptive case.

Computing an ordinary derivative does not require an ultrafilter, and symbolic algebra also obtains these values. The contribution is the proof connecting exact executable syntax, completion-dependent domain conditions, and an ordinary output that is invariant over the remaining completions.

2.3 Elementary Functions and Vector Derivatives

The polynomial result uses a complete rational extractor. A second language extends differentiation while returning symbolic real expressions. Its syntax has rational constants, finitely many real variables, arithmetic including general division, negation, and $\sin$, $\cos$, $\exp$, $\log$, and $\sqrt{\cdot}$. A program $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a finite vector of these expressions. The domain predicate is compositional: denominators are nonzero, logarithm and square-root arguments are positive, and all subexpressions meet their own conditions. These are sufficient smoothness conditions. In particular, this language excludes the square-root boundary at zero.

The executable compiler $J(F, v)$ applies the usual chain rules and produces expression syntax for a Jacobian-vector product. Tangent expressions are evaluated at the base point and held fixed as the scalar step varies. Coordinate basis directions give the Jacobian, and a scalar output gives a gradient. The compiler receives no derivative or convergence certificate.

Theorem 2 (Compiled elementary quotient). *Let F and v be represented by the differentiable language and let a satisfy the source and tangent domain conditions. The generated expression is defined at a and evaluates to $DF(a)v(a)$. For every compatible completion $\mathcal{U}$ and sequence h satisfying $h \rightarrow_{\mathcal{U}} 0$ and $\{k : h(k) \neq 0\} \in \mathcal{U}$,*

$$\text{st}_{\mathcal{U}} \left(\frac{F(a + h v(a)) - F(a)}{h} \right) = J(F, v)(a) = DF(a)v(a)$$

componentwise. The perturbed source expressions satisfy their domain conditions eventually in $\mathcal{U}$.

Proof. Induction on expressions proves Fréchet differentiability on the specified domain, correctness of the actual derivative compiler along differentiable input curves, and preservation of the domain by compilation. The domain is open. Applying the compiler theorem to $t \mapsto a + t v(a)$ identifies its value with $DF(a)v(a)$. The ordinary derivative theorem gives

convergence of the literal difference quotient along the punctured neighborhood of zero. The two hypotheses on h provide a map into that filter, giving the claimed limit in each completion. Openness gives eventual validity of the perturbed operations. No infinitesimal term is declared zero before division. $\square$

The modules `DifferentiableCore`, `Differentiable`, and `DifferentiableInfinitesimal` implement this construction. The declarations `Expr.jvp_eq_fderiv`, `Expr.domain_jvp`, and `Expr.quotient_standardPart` establish its three main obligations. The Python adapter returns immutable expression syntax. Kernel verification proves that this actual output agrees with the formal compiler before instantiating its semantic theorem. Supplying a rational point additionally requests a kernel proof of the source, tangent, and result domains there. The domain tactic is incomplete. Failure leaves an obligation unresolved, without asserting that a derivative or standard part does not exist. Floating-point samples are labelled approximations and carry no error certificate.

For a concrete example, take

$$F(x, y) = \left(\sin(x)e^y, \log(1 + x^2 + y^2), \frac{\sqrt{1 + x^2}}{1 + y^2} \right), \quad a = (1, 0), \quad v = (1, 2).$$

The module `DifferentiableExamples` proves

$$DF(a)v = \left(\cos(1) + 2 \sin(1), 1, \frac{1}{\sqrt{2}} \right).$$

It then reuses the observation-dependent step from the preceding case. Either the observation establishes nonzeroness of the original step or the program replaces it by $1/n^2$. Every completion of either accepted trace gives this same vector standard part. This connection reuses the residue observation engine's step guarantee. It does not add elementary comparisons to that engine or pass an elementary quotient to Laurent normalization.

The two computational guarantees have distinct scopes. Laurent extraction decides whether every compatible completion shares a finite standard part, including certified rejection. The elementary compiler proves a derivative and quotient identity under explicit domains, with symbolic output. It supplies neither complete elementary limit decisions nor reverse-mode differentiation. Arbitrary host-language control flow and tensor execution are also outside this extension.

3 Finite Observations and Classical Completion

The formal semantics uses total real sequences, $\text{Seq} = \mathbb{N} \rightarrow \mathbb{R}$, and their comparison sets. For $x, y \in \text{Seq}$, write $L_{x,y} = \{n : x(n) < y(n)\}$ and $E_{x,y} = \{n : x(n) = y(n)\}$. A positive comparison observation selects its comparison set. A negative observation selects its complement. The family $\Gamma \subseteq \mathcal{P}(\mathbb{N})$ records these actual selected sets, rather than Boolean names for them. The Lean definitions are `Sequence`, `comparisonLt`, `comparisonEq`, and `Commitments`.

A free ultrafilter $\mathcal{U}$ on $\mathbb{N}$ is a proper filter that decides every subset and contains no finite set. Equivalently, it extends the cofinite filter $\mathcal{F}_{\text{cof}} = \{A \subseteq \mathbb{N} : A^c \text{ is finite}\}$. A completion of Γ is such an ultrafilter containing every member of Γ . Lean's `Completion` records an ultrafilter, its extension of the cofinite filter, and containment of the commitments. `Extendible` asserts that a completion exists. This is an existence specification, not an algorithm constructing an ultrafilter.

The usual ultrapower interpretation identifies sequences x and y when $E_{x,y} \in \mathcal{U}$, yielding the hyperreal field $\mathbb{R}^{\mathbb{N}}/\mathcal{U}$ [13]. We use this interpretation to explain the observations

and standard parts. The general project semantics formalizes sequence comparisons, filter limits, and completion properties. The worked quotient additionally uses Mathlib's existing hyperreal field. It does not reconstruct the whole ultrapower field or prove a new transfer theorem. Different completions can give different comparison answers to the same sequence representatives.

Definition 1 (Free finite-intersection property). The family Γ has the free finite-intersection property when every finite subfamily of $\Gamma \cup \mathcal{F}_{\text{cof}}$ has nonempty intersection. The intersection of the empty family is $\mathbb{N}$. This is Lean's `HasFreeFIP`.

Theorem 3 (Completion criterion). For any family $\Gamma \subseteq \mathcal{P}(\mathbb{N})$,

$$\text{HasFreeFIP}(\Gamma) \iff \text{Extendible}(\Gamma).$$

If Γ is finite, these conditions are also equivalent to infinitude of its joint intersection $J_\Gamma = \bigcap_{A \in \Gamma} A$.

Proof. A completion contains every finite intersection of commitments and cofinite sets. Such an intersection is nonempty because the filter is proper. Conversely, the free finite-intersection property makes the filter generated by $\Gamma \cup \mathcal{F}_{\text{cof}}$ proper. The classical ultrafilter lemma extends it to an ultrafilter [10]. This extension is free: a finite member and its cofinite complement would otherwise have empty intersection.

Now suppose Γ is finite. If J_Γ is infinite, intersecting it with any cofinite set leaves a nonempty set, which supplies the required finite-intersection property. If J_Γ is finite, adjoining its cofinite complement to Γ gives an empty finite intersection, contradicting that property. $\square$

Lean proves the general equivalence as `hasFreeFIP_iff_extendible` and the finite-family characterization as `jointlyInfinite_iff_extendible`. Finiteness is a property of the executable observation traces, not a necessary hypothesis of the general completion theorem. Propositional satisfiability alone is insufficient: distinct atoms can denote disjoint actual sets even when their conjunction has a Boolean model. An implementation must connect its representations to these sets and preserve the semantic invariant.

3.1 Standard Parts as Filter Limits

For a sequence x , completion $\mathcal{U}$, and $c \in \mathbb{R}$, define

$$\text{NearStandardAt}(\mathcal{U}, x, c) \iff \forall \varepsilon > 0, \{n : |x(n) - c| < \varepsilon\} \in \mathcal{U}.$$

This says that x converges to c along $\mathcal{U}$. In the ultrapower interpretation, $x - c$ is infinitesimal and c is the standard part of the represented hyperreal. Lean defines `NearStandardAt` with `Filter.Tendsto` and proves uniqueness as `nearStandardAt_unique`.

Proposition 1 (Ordinary limits are completion invariant). If $x(n) \rightarrow c$ ordinarily, then $\text{NearStandardAt}(\mathcal{U}, x, c)$ for every free completion $\mathcal{U}$.

Proof. For every $\varepsilon > 0$, ordinary convergence makes $\{n : |x(n) - c| < \varepsilon\}$ cofinite. Every free completion contains this set. $\square$

The Lean theorem is `cofinitelimit_completion_invariant`, where `CofiniteLimit` denotes convergence along the cofinite filter. Its hypothesis must be proved for the actual sequence. The return of a numerical candidate does not establish it. The verified residue extractor below proves a support-relative version directly from exact finite coefficient checks. A successful result is shared by every completion consistent with the current observations. Failure to find a shared result need not mean that individual completions lack standard parts.

3.2 A Choice That Remains Partial

Let $a(n) = (-1)^n$. Initially both the even and odd index classes are infinite, so either comparison choice is compatible with a free completion. Every completion selects one parity, giving a standard part 1 or -1 , but there is no common value across all completions. The verified extractor therefore returns `None` on universal support. Accepting $a < 0$ retains the odd class. Extraction then returns -1 in every compatible completion, and an attempted commitment $a = 1$ is rejected. Extraction itself makes no further choice.

The two observations $a < 0$ and $a = 1$ are individually realizable but select disjoint parity classes. Their conjunction could be satisfied if they were represented only by independent Boolean names, yet their actual sets admit no common completion. Lean proves this obstruction in `Counterexample`. This is why the executable state must represent the selected index sets and check their joint support.

Selecting the odd class does not make all sufficiently large indices odd, nor does it identify a unique ultrafilter. It requires only that the odd class belong to the completion. This distinction motivates the runtime: it retains all currently compatible residue alternatives and returns an invariant result when those alternatives agree, without computing a complete ultrafilter.

4 Verified Periodic Laurent Core

4.1 Syntax and Exact Normalization

The executable grammar contains exact rational constants, n , $1/n$, finite periodic rational tables, addition, subtraction, multiplication, and division by an explicitly specified monomial cn^k , where $c \in \mathbb{Q} \setminus \{0\}$ and $k \in \mathbb{Z}$. A table $(b_0, \dots, b_{p-1})$ denotes $b_{n \bmod p}$. $(-1)^n$ is the table $(1, -1)$. Validity checking rejects empty tables and zero divisor coefficients. The coefficient periods of expressions are combined by least common multiples. n and $1/n$ have coefficient period one despite not being periodic sequences. On each residue of the resulting coefficient period, the normalizer produces

$$\frac{p(n)}{n^s}, \quad p(X) = a_0 + a_1X + \dots + a_dX^d, \quad a_i \in \mathbb{Q}, \quad s \in \mathbb{N}.$$

The numerator is a dense list of exact rational coefficients, evaluated by Horner's rule. Every coefficient is retained, including terms that a later division can shift into the constant term. There is no series truncation. The refinement theorem in `ResidueExpr` proves agreement with the source sequence at every positive index, including when residues are enumerated at a multiple of the expression's period. Lean's total inverse at index zero is irrelevant to free-ultrafilter semantics.

4.2 Comparison Masks with Computed Cutoffs

The executable sign recursion returns both a sign and a natural cutoff $K \geq 1$. If a Horner tail has nonzero leading coefficient r and cutoff K , adjoining the constant coefficient a uses

$$K' = \max \left\{ K, \left\lceil \frac{|a|}{|r|} \right\rceil + 1 \right\}.$$

The proof maintains the tail's sign and a magnitude of at least $|r|$ at every real argument beyond the cutoff. Zero tails are handled separately, including representations with trailing zero coefficients. Division by n^s preserves the sign for $n \geq K$. The comparison compiler applies this rule to the difference of two normalized forms on each residue of the operands' common period and takes the maximum cutoff. Thus the reported mask agrees with the actual comparison at every later index. No supplied correctness field or sampled evaluation substitutes for this proof.

4.3 Support Refinement

The state is a Boolean support table of positive length with at least one true entry. If its period is p and a selected comparison mask has period q , the transition enumerates $\ell = \text{lcm}(p, q)$ residues and computes

$$S'(r) = S(r \bmod p) \wedge M(r \bmod q), \quad 0 \leq r < \ell.$$

It accepts exactly when some entry remains true. The proved carrier identity $\text{carrier}(S') = \text{carrier}(S) \cap \text{carrier}(M)$ shows that period growth preserves every prior commitment. A rejected transition leaves the API state unchanged. In particular, compatible choices modulo 2 and 3 can select one residue modulo 6, whereas incompatible choices at periods with shared factors are rejected.

4.4 Shared Standard Parts

Standard-part extraction performs a separate, read-only check at the least common multiple of the support period and the queried expression's period. On each active residue, it requires every numerator coefficient above degree s to vanish, then reads the coefficient of degree s (zero if absent). It returns a rational only if these finite limits agree across all active residues. Inspecting the enlarged common period does not alter the stored support. Failure may mean divergence on an active branch or disagreement between branches. It need not mean that a standard part fails to exist in individual completions.

Theorem 4 (Accepted traces and extracted standard parts). *Let a finite trace T be accepted from universal support, with final carrier $S_T \subseteq \mathbb{N}$, and let Γ_T be the family of its actual selected comparison sets, using complements for negative choices. Then Γ_T has a free completion, and for every free ultrafilter $\mathcal{U}$,*

$$S_T \in \mathcal{U} \iff \forall A \in \Gamma_T, A \in \mathcal{U}.$$

If extraction on S_T returns $q \in \mathbb{Q}$ for an expression e , then $\lim_{\mathcal{U}} e(n) = q$ in every free completion of Γ_T .

Proof. Each computed mask differs from its actual selected set only on a finite prefix, so the two sets have identical membership in every free ultrafilter. The carrier identity for least-common-multiple refinement and induction over accepted intersections give the displayed equivalence. A nonempty support of positive finite period is infinite, which supplies a free completion. Successful extraction proves the same ordinary limit for every active normalized form. There are finitely many such forms, so their eventual bounds can be combined and then specialized to the actual residue of each index in S_T . Restricting to any free ultrafilter containing S_T preserves the resulting limit, and the equivalence applies it to every completion of the trace. $\square$

The support and expression proofs are in `ResidueSupport` and `ResidueExpr`. Comparison correctness is in `ResidueComparison`. Transition, extraction, and trace proofs are in `ResidueRuntime`, `ResidueLimit`, and `ResidueTrace`. These reuse the scalar Laurent normalization, sign, and limit proofs. In particular, `run_standardPart_sound` concludes correctness for every completion of the recorded observations. Successful extraction on universal support also implies ordinary cofinite convergence.

Theorem 5 (Completeness of shared extraction). *Let e be a valid expression and S a nonempty residue support. For each $q \in \mathbb{Q}$,*

$$\widehat{\text{st}}_S(e) = q \iff \forall \mathcal{U} \supseteq \mathcal{F}_{\text{cof}}, \quad S \in \mathcal{U} \implies \lim_{\mathcal{U}} e(n) = q.$$

Moreover, if the limit is shared for any $r \in \mathbb{R}$, then r is rational and the extractor returns it. A failed extraction therefore rules out every shared finite real value, not only the rational candidates searched by the implementation.

Proof. Soundness was proved above. For the converse, each active residue of the support/expression common period is infinite and supports a free ultrafilter containing S . Restricting to that residue identifies e with one rational Laurent form. A scalar Laurent form with a nonzero positive power of n has unbounded absolute value along every free filter. Otherwise its limit is its rational constant term. A common finite limit consequently forces every active residue to have the same rational constant term, exactly the finite checks performed by the extractor. $\square$

The declarations `standardPart_complete_real`, `standardPart_iff_all_completions`, and `standardPart_none_no_common_real` in `ResidueLimitCompleteness` prove the converse and its failure consequence. Validity and nonempty support are essential hypotheses. Invalid syntax is not a divergent sequence, and an empty set of completions would make a universal limit claim vacuous. The equivalence with trace completions in Theorem 4 transfers completeness to accepted traces.

The executable diagnostic returns either the common rational, an active residue whose normalized form diverges, or two active residues with different finite limits. Residues are reported modulo the support/expression common period. A separate invalid-input result keeps malformed syntax and empty support distinct from mathematical failure. The declarations `standardPart_none_iff_diagnostic_failure`, `diagnoseStandardPart_divergent_completion`, and `diagnoseStandardPart_disagreement_completions` in `ResidueLimitDiagnostic` prove that these failure cases are exhaustive and exhibit compatible completions realizing them. The classification explains why a shared standard part is unavailable on the current support.

4.5 Implementation Boundary

The native `residue_checker`, built from `ResidueChecker`, calls these same executable functions. The Python interface constructs expression trees and records observations. Ordinary native calls rely on faithful transport, JSON parsing, native compilation, and execution. The optional replay path below checks individual exported claims without trusting the native answer. The implementation enumerates its support tables explicitly, so they can grow to the least common multiple of introduced periods. No period minimization or general scalability guarantee is claimed.

5 Adaptive Execution and Fixed Completions

A trace theorem explains a completed list of observations. To connect that theorem to control flow, `ObservationPrograms` defines a finite program tree. A leaf returns a residue expression. A query node contains a strict comparison or equality and two continuation programs. Later queries and the returned expression may therefore depend on earlier answers. The syntax describes finite decision trees, not arbitrary Python programs or unbounded loops.

The executable interpreter consumes a list of proposed Boolean answers. At each query it forms the actual observation, calls the existing verified commitment operation, and follows

the selected continuation only if that commitment succeeds. Missing answers or rejected commitments cause execution failure. A leaf checks expression validity and nonempty support, then performs read-only standard-part extraction. An accepted execution may therefore have a leaf result of `None`. This is different from failing to execute the program.

The result records the selected trace, final support, leaf expression, extracted value, and unused proposed answers. These are data, not supplied proofs of consistency. The execution proof first shows that replaying the recorded trace with the existing residue transition function gives the recorded support. It then applies the established trace-to-completion theorem.

Theorem 6 (Fixed-completion realization). *Suppose a finite program executes successfully from universal support. There exists a free ultrafilter $\mathcal{U}$ such that interpreting every comparison by membership of its actual comparison set in $\mathcal{U}$ follows the same branches and selects the same leaf expression. In fact, every free completion of the recorded trace selects that leaf. If the extracted value is $q \in \mathbb{Q}$, the leaf expression has standard part q in every such completion.*

Proof. Induction on the executable program connects each accepted query to its recorded observation and its chosen continuation. The trace theorem supplies a free completion containing every selected observation. For a positive answer the comparison set belongs to that completion. For a negative answer its complement belongs, so the comparison set does not. A second induction therefore follows the same continuation at each node under the fixed ultrafilter interpretation. The support characterization and extraction theorem give the final standard part. $\square$

The corresponding Lean declarations are `Program.execute_interpret`, `Program.execute_realized`, and `Program.execute_standardPart_sound`. Corollary 1 instantiates these declarations with an observation-dependent denominator obligation and proves a common result for every accepted input stream. Further kernel-checked examples exercise different later queries, rejected contradictory choices, exhausted choice streams, and accepted unknown outputs.

The quantifiers matter. Each accepted run admits a fixed completion. Incompatible alternative runs need not share one. Nor does the interpreter compute the completion used in the proof. Its proposed answers can come from a caller or search procedure, but acceptance depends on the semantic check. Only the visited queries and returned expression are validated. A common ordinary result across alternative runs requires an additional argument about their extracted values, as supplied by `accepted_result` in the domain example. Fixed-completion realization alone does not imply that all branches compute the same value.

For a continuing mathematical history, there is a related classical observation. If $\Gamma_0 \subseteq \Gamma_1 \subseteq \dots$ and every Γ_k has a free completion, then $\bigcup_k \Gamma_k$ has one free completion. Every finite subfamily of the union lies in a sufficiently late stage, giving the free finite-intersection property required by Theorem 3. Lean proves this as `extendible_iUnion_of_monotone`. This is a compactness consequence of the existing completion criterion. It neither supplies an executable ultrafilter nor certifies future behavior from one finite snapshot. A running system needs an invariant covering every accepted step.

6 Kernel Replay of Exported Sessions

The public API records the exact operand syntax, comparison operation, and selected truth value of each accepted commitment. Rejections and read-only probes do not enter this trace. An extraction snapshot freezes the accepted trace T , current support S , query expression e , and claimed result $v \in \text{Option}(\mathbb{Q})$. Later state changes do not alter it. The exported bundle contains exact rational JSON data, generated Lean declarations, and a manifest associating their hashes with the pinned proof sources.

The Lean structure `Residue.Replay.Snapshot` contains these data and no fields asserting their correctness. Its Boolean check requires a valid expression, nonempty support, and the two executable equalities

$$\text{run}(\text{universe}, T) = \text{some}(S), \quad \text{standardPart}(S, e) = v.$$

For a closed exported snapshot σ , generated source proves $\text{check}(\sigma) = \text{true}$ using `decide + kernel`. The theorems `Snapshot.trace_extendible` and `Snapshot.support_mem_iff` then establish existence of a compatible free completion and the exact final-support characterization. If $v = \text{some}(q)$, `Snapshot.standardPart_sound` yields the standard part q in every completion of that actual exported trace. For $v = \text{none}$, `Snapshot.no_common_standardPart` excludes a common finite real standard part, and `Snapshot.failure_classified` proves that the computed diagnosis is divergence or disagreement. Replay does not check the particular witness fields returned separately by the native diagnostic interface. Individual completions can still have standard parts, and later observations may select a value.

Verification regenerates fixed Lean syntax from validated snapshot data, checks artifact association, refreshes the imported build, and elaborates the regenerated source under a resource deadline. Every generated theorem has an axiom report. The accepted set is `propext`, `Classical.choice`, and `Quot.sound`. The replay uses neither `native_decide` nor `native_evaluation` as proof evidence: those shortcuts would add compiler trust beyond kernel reduction. The dependency libraries, their stated axioms, the Lean kernel, and the surrounding operating system and hardware remain part of the assurance assumptions.

For a successfully replayed claim, a wrong native answer cannot supply the checked equalities. Replay nevertheless proves a statement about the exported syntax, not that Python captured the intended expression or that the recorded observations occurred in an external process. Capture, wire-format mapping, and source-data association are tested implementation obligations. Hashes bind artifact components for checking. They do not authenticate the historical truth of the session. This distinction separates a kernel-checked mathematical instance from a formally verified end-to-end application.

6.1 Validation of the Execution Boundary

The integration tests exercise exact large rationals, terms retained through later division, changes of period, incompatible commitments, and extraction without mutation. Replay tests include malformed snapshots, invalid expressions claiming an unknown result, inconsistent final supports, altered artifacts, and incorrect native answers. In one regression, a mocked native checker reports 2 for the constant expression 1. Snapshot capture records that claim, but kernel replay rejects it. These tests check concrete interface behavior. The general all-indices and all-completions guarantees come from the Lean theorems, whose audited dependencies are restricted to the stated axiom set.

7 Related Work

Beeson [2] used nonstandard analysis in Mathpert to justify symbolic calculus operations and proved correctness of an implemented infinitesimal-elimination procedure. His treatment also makes the side conditions of symbolic operations explicit. This is a close precedent for computational infinitesimals with correctness obligations. Our focus is the refinement of a finite support representing compatible completions through accepted observations, together with results that hold in every completion remaining after the trace.

Fleuriot and Paulson [6] constructed the hyperreals and mechanized nonstandard real analysis in Isabelle/HOL. Dou and Yu [5] formalized the filter extension principle in Coq. Mathlib [14] already supplies filters, ultrafilters, limit theory, and a quotient construction

of the hyperreals. Our development uses its filter and analysis infrastructure and states its semantic results for arbitrary compatible free ultrafilters. The mathematical existence of completions is established background. The implementation-specific obligation is to connect executable normalization, comparisons, state updates, and extraction to that semantics.

Nonstandard analysis also supports program semantics and verification. Benveniste et al. [4] give a nonstandard semantics of hybrid-system modelers, and Hasuo and Suenaga [9] investigate nonstandard static analysis of hybrid systems. Kido, Chaudhuri, and Hasuo [11] extend abstract interpretation with infinitesimals, proving soundness and termination under uniform widening and evaluating a prototype. The present artifact addresses a smaller expression language and an explicit finite observation trace. It does not provide a hybrid-system analyzer or a verified semantics for general programs.

Automatic differentiation [1, 7] and computer algebra systems such as SymPy [12] provide established ways to obtain the ordinary derivative in the worked example. Our question concerns the interpretation of a program whose observations and intermediate infinitesimal quantities remain underdetermined. The result 12 is elementary. The additional obligation is to justify the selected quotient in a fixed completion and its standard part in every completion compatible with the run. We claim no computational advantage over differentiation or symbolic algebra.

For the verified fragment, a support mask is an exact finite representation of the remaining residue alternatives. Intersection after lifting to a common period performs constraint refinement, so an ordinary finite-domain constraint solver can represent the same alternatives. The mathematical contribution of this representation is its proved correspondence with free-ultrafilter completions and the universal guarantee for a successfully extracted standard part.

Beeson and Wiedijk [3] use filters to give semantics to symbolic calculations with infinity, including partially informative outcomes and refinement. Their work is a direct precedent for interpreting calculations through filters. Our particular interface records accepted observations about index sets, proves that its finite executable states describe exactly the compatible free completions, and relates adaptive execution and extracted outputs to those completions.

Harrison and Théry [8] separate external solution computation from formal checking in their integration of HOL and Maple. Kernel replay follows that established architecture. The statements checked here concern a recorded observation trace, its compatible completions, and a completion-invariant standard part. The general idea of checking an externally obtained symbolic result is not a novelty claim.

The contribution is the composition of three obligations: executable observations preserve a nonempty class of classical completions, one completion reproduces the control flow of each accepted adaptive run, and successful extraction is valid in every completion of that run. Completeness characterizes exactly when that ordinary value exists across all remaining completions. Generic polynomial differentiation, the elementary compiler, and the domain example connect this interface to literal infinitesimal quotients. The compiler rules themselves are standard differentiation rules. This combination makes the witness-free execution claim precise for the implemented language. It does not make finite-domain refinement, filter semantics, or infinitesimal differentiation new in themselves. Reusing the pattern with another non-constructive object would require a representation of finite observations, an executable consistency check connected to an extension theorem, and a sound extraction operation. The present artifact supplies one such instance, not a formal abstraction covering all such objects.

8 Artifact Evidence and Boundaries

The main evidence is the Lean proof chain from executable observations to classical completions, adaptive execution, and standard-part extraction. The project audit rebuilds the development, rejects unfinished proofs and project-local axioms, and checks the reported dependencies of the maintained theorem roots. The allowed classical dependencies are `propext`, `Classical.choice`, and `Quot.sound`. The audit includes the operational semantics and the infinitesimal case. The source-to-theorem links distinguish the general completion criterion, executable refinement, program realization, and the concrete calculus identity.

The artifact combines universal theorems with concrete replay. Polynomial differentiation covers arbitrary rational coefficients, rational points, nonzero rational scales, and positive integer step powers. Every accepted domain-program run establishes a nonzero denominator and returns 12. Three saved snapshots check both branches and the absence of a common finite value before the observation. Supplementary forward and backward cubic snapshots exhibit distinct infinitesimal errors with the same standard part.

The proof boundaries are concrete. Complete extraction covers exact rational coefficients, finite coefficient periods, and monomial division. Representing an inverse on selected support does not implement complete rational-function normalization. The elementary derivative language supports general division and five elementary primitives on explicit smooth domains, but supplies no complete comparison or limit algorithm for them. Arbitrary sequence predicates remain outside both executable interfaces. Ordinary native calls assume faithful transport, parsing, compilation, and execution. Replay checks a regenerated mathematical instance, but faithful capture of the intended external computation remains unverified. The finite program syntax has no loops, and the Python API is not formally translated to it. Extraction completeness concerns values shared across every compatible completion, not the selection of a preferred completion.

The next boundary is faithful capture, richer observations, and efficient differentiation. A verified translation from the public API to finite program syntax would address capture. General rational functions would require a verified denominator check and exact comparison and limit procedures. More general observations would likewise require executable checks with their own correctness arguments. The classical assertion that one consistent polarity exists does not supply such a procedure.

9 Conclusion

A finite computation can use infinitesimals through observations whose consistency has been checked, while its ultrafilter interpretation remains non-constructive. The executable state records the observations. The completion theorem supplies their classical interpretation. The program theorem shows that accepted adaptive execution is compatible with one fixed completion, and sound and complete extraction identifies exactly the finite values shared by all remaining completions.

Generic polynomial differentiation connects that interface to exact literal quotients and the ordinary derivative. The adaptive case makes the semantic obligation visible: an infinitesimal may be zero, so an observation must establish invertibility or cause the program to choose a different step. Both branches then extract the same ordinary value. The implementation never constructs the ultrafilter that justifies the selected execution. The elementary compiler extends the quotient guarantee to differentiable vectors with symbolic real outputs. Complete extraction remains specific to the periodic Laurent language. Faithful capture, broader observations, and efficient derivative execution remain to be established.

Artifact and Reproducibility

The repository contains the Lean modules under `Hyperreals`, Python adapters under `src/hyperreals`, and pinned Lean, Mathlib, and Python 3.14 configurations. Install the locked development dependencies with `uv sync --locked --extra dev`, then run `make lean-audit` and `uv run --locked --extra dev pytest`. The elementary vector example is reproduced and kernel-checked by `scripts/differentiable_case.py`. The domain example is reproduced by `scripts/domain_infinitesimal_case.py`, and the elementary cubic calculations by `scripts/in_finitesimal_case.py`. Their saved snapshots can also be checked individually with `scripts/verify_replay.py`. The example README records the exact commands and the distinction between native output and kernel replay. Supplementary exact-arithmetic comparisons and a delayed-choice example are documented under `benchmarks` and `examples/delayed-choice`. Their measurements support no scaling or performance-ranking claim.

The repository is github.com/krzysztofwoś/hyperreals. A submission should archive the exact accompanying revision. A moving repository URL does not fix the evaluated artifact.

Acknowledgements

I thank Stephen Fitz for discussions about finite symbolic computation and the ontological status of the continuum, which motivated the question of how a finite interface can be justified by a classical completion theorem.

References

- [1] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18 (153):1–43, 2018. URL <http://jmlr.org/papers/v18/17-468.html>.
- [2] Michael Beeson. Using nonstandard analysis to ensure the correctness of symbolic computations. *International Journal of Foundations of Computer Science*, 6(3):299–338, 1995. doi: 10.1142/S0129054195000172. URL <https://www.michaelbeeson.com/research/papers/nsappt.pdf>.
- [3] Michael Beeson and Freek Wiedijk. The meaning of infinity in calculus and computer algebra systems. *Journal of Symbolic Computation*, 39(5):523–538, 2005. doi: 10.1016/j.jsc.2004.12.002. URL <https://www.cs.ru.nl/~freek/pubs/limits.pdf>.
- [4] Albert Benveniste, Timothy Bourke, Benoît Caillaud, and Marc Pouzet. Non-standard semantics of hybrid systems modelers. *Journal of Computer and System Sciences*, 78(3):877–910, 2012. doi: 10.1016/j.jcss.2011.08.009. URL <https://www.sciencedirect.com/science/article/pii/S002200011001061>.
- [5] Guowei Dou and Wensheng Yu. Formalization of the filter extension principle (FEP) in Coq, 2024. URL <https://arxiv.org/abs/2407.06222>.
- [6] Jacques D. Fleurbaey and Lawrence C. Paulson. Mechanizing nonstandard real analysis. *LMS Journal of Computation and Mathematics*, 3:140–190, 2000. doi: 10.1112/S1461157000000267.
- [7] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, 2nd edition, 2008. doi: 10.1137/1.9780898717761.
- [8] John Harrison and Laurent Théry. A skeptic’s approach to combining HOL and Maple. *Journal of Automated Reasoning*, 21:279–294, 1998. doi: 10.1023/A:1006023127567.
- [9] Ichiro Hasuo and Kohei Suenaga. Exercises in nonstandard static analysis of hybrid systems. In P. Madhusudan and Sanjit A. Seshia, editors, *Computer Aided Verification – CAV 2012*, volume 7358 of *Lecture Notes in Computer Science*, pages 462–478. Springer, 2012. doi: 10.1007/978-3-642-31424-7_34. URL https://doi.org/10.1007/978-3-642-31424-7_34. Extended version with proofs available at <https://group-mmm.org/ichiro/papers/cav2012Extended.pdf>.

- [10] Thomas Jech. *Set Theory: The Third Millennium Edition, Revised and Expanded*. Springer Monographs in Mathematics. Springer, 2003. doi: 10.1007/3-540-44761-X.
- [11] Kengo Kido, Swarat Chaudhuri, and Ichiro Hasuo. Abstract interpretation with infinitesimals: Towards scalability in nonstandard static analysis (extended version), 2015. URL <https://arxiv.org/abs/1511.00825>.
- [12] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017. doi: 10.7717/peerj-cs.103. URL <https://peerj.com/articles/cs-103/>.
- [13] Abraham Robinson. *Non-Standard Analysis*. North-Holland, 1966.
- [14] The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824. URL <https://doi.org/10.1145/3372885.3373824>.